Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University(SPPU)
ISO 21001:2018 Certified by TUV SUD

Sustainability Development Goals Mapping

Department: Computer Engineering **A.Y.: 2025-26**
Sem: 2
Subject: Full Stack Development Lab.

Mini Project Title: AgroMart – Farm to Table Marketplace.

NAME	Piyush Ahirao	Tanishq Bharti
PRN	123B1B077	123B1B092

Class and Div: T.Y. B. Lab Location: 6218LA

Sr. No.	SDG No.	SDG	Description
1	SDG 2	Zero Hunger	AgroMart connects farmers directly with consumers, improving access to fresh and quality food.
2	SDG 8	Decent Work & Economic Growth	It helps farmers increase their income by providing a platform for direct selling without intermediaries.
3	SDG 9	Industry, Innovation & Infrastructure	The system uses modern web technologies to build an efficient digital agricultural marketplace.
4	SDG 10	Reduced Inequalities	It provides equal opportunities for small and local farmers to reach a wider market.
5	SDG 12	Responsible Consumption & Production	AgroMart encourages direct sales, reducing food wastage and promoting better consumption practices.

Piyush Ahirao

Student Name and Signature

S. B. Galande

Mr.Shailesh B. Galande

Project guide Name & Signature

Scanned with OKEN Scanner

PRELIMINARY MINI PROJECT REPORT ON

"AgroMart – Farm to Table Marketplace"

SUBMITTED TOWARDS THE PARTIAL FULFILLMENT OF
THE REQUIREMENTS OF
BACHELOR OF TECHNOLOGY (Third Year B. Tech.)
Academic Year: 2025-26

By:
1. **Tanishq Bharti [123B1B092]**
2. **Piyush Ahirao [123B1B077]**

Under The Guidance of

Prof.Shailesh B. Galande

DEPARTMENT OF COMPUTER ENGINEERING,
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
SECTOR 26, NIGDI, PRADHIKARAN, PUNE – 411044

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that, the mini project entitled

"AgroMart – Farm to Table Marketplace"

is successfully carried out as a mini project for Full Stack Development Laboratory and successfully submitted by following students of "PCET'S Pimpri Chinchwad College of Engineering, Nigdi, Pune-44".

Under the guidance of Prof. Shailesh B. Galande.

In the partial fulfillment of the requirements for the Third Year B. Tech.

(Computer Engineering)

By:

1. **Tanishq Bharti [123B1B092].**
2. **Piyush Ahirao [123B1B077].**

Prof._____
Project Guide.

ABSTRACT

AgroMart – Farm to Table Marketplace is a web-based application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) with the objective of connecting farmers directly with customers by eliminating intermediaries. In the traditional agricultural system, farmers often depend on middlemen, which results in lower profits for farmers and higher prices for consumers. This system provides a digital solution where farmers can register, upload their products, and manage orders, while customers can browse products by categories, add items to the cart, and place orders easily. The platform includes three main users: Farmer, Customer, and Admin, where the admin verifies farmer details before allowing them to sell on the platform. Additional features such as product management, order tracking, user dashboards, a basic chatbot for handling common queries, and simulated document verification are also included. MongoDB is used for data storage, while Node.js and Express.js handle backend operations and React.js is used for the frontend interface. AgroMart provides a simple, user-friendly, and efficient solution for digital agricultural marketing, helping farmers receive fair prices and enabling customers to purchase fresh products directly from farms.

INDEX

Chapter	Contents	Page No.
1	Introduction	7
1a	Problem Statement	7
1b	Project Idea	7
1c	Motivation	8
1d	Scope	8
1e	Literature Survey / Requirement Analysis	9
2	Project Design	10
2a	Hardware, Software & Resource Requirements	10
2b	Dataset Design	11
2c	Hours Estimation	12
3	Module Description	13
3a	Farmer Module	13
3b	Customer Module	14
3c	Admin Module	14
3d	AI-Based Features	15
3e	Block Diagram with Explanation	15
3f	Comparative Study	19
4	Results & Discussion	20
4a	Source Code	20
4b	Test Cases	23
4c	Screenshots & GUI Description	27
5	Conclusion & Future Scope	31
	References	32
	Annexure – Plagiarism Report	33

a)LIST OF FIGURES & TABLES

Figure No.	Figure	Page No.
Figure 1	System Architecture Diagram (Description)	15
Figure 2	Use Case Diagram	15

Table No.	Figure / Table Name	Page No.
Table 1	Hardware Requirements	9
Table 2	Software Requirements	9
Table 3	MongoDB Collection – Users	11
Table 4	MongoDB Collection – Products	11
Table 5	MongoDB Collection – Orders	12
Table 6	Hours Estimation Table	14
Table 7	Test Cases – Authentication	20
Table 8	Test Cases – Product Management	21
Table 9	Test Cases – Order Placement	21
Table 10	Comparative Study – Existing Platforms	18

CHAPTER 1: INTRODUCTION

1a) Problem Statement

Agriculture is one of the most important sectors in India, but farmers face challenges such as low income, lack of direct market access, and dependency on intermediaries. In the traditional system, products pass through multiple middlemen, which reduces farmers' profits and increases prices for consumers. There is also a lack of transparency in pricing and quality due to long supply chains.

The main problems are:

- Farmers do not have a direct platform to sell products
- Prices are not transparent
- Middlemen increase product cost
- Product quality reduces due to multiple handling

Therefore, there is a need for a system that connects farmers directly with consumers and ensures fair and transparent agricultural marketing.

1b) Project Idea

AgroMart – Farm to Table Marketplace is a web-based platform developed using the MERN stack that connects farmers directly with consumers. The main idea of the system is to eliminate middlemen in the agricultural supply chain so that farmers can sell their products at fair prices and customers can purchase fresh produce directly from farms.

In this platform, farmers can register by providing their personal details, farm information, and necessary documents such as identity proof and land ownership proof. These details are verified by the admin to ensure that only genuine farmers are allowed to sell products. Once approved, farmers can add their products with details like image, description, price in rupees, and available quantity.

Customers can browse products by categories, search for items, view product details along with farmer information, and add products to the cart. They can then place orders using available payment options such as Cash on Delivery and online payment methods. The system also provides order tracking to help users know the status of their orders.

The platform includes three main users: Farmer, Customer, and Admin. Farmers can manage their products and orders through a dashboard, customers can easily purchase products, and the admin can verify users and manage the system. Additional features such

as a simple chatbot for answering user queries and document verification simulation are included to improve user experience.

1c) Motivation

The motivation behind developing AgroMart is to solve the common problems faced by farmers in selling their agricultural products. In the traditional system, farmers depend on middlemen to sell their produce, which results in lower profits for farmers and higher prices for customers. This creates an imbalance in the agricultural market.

With the increasing use of the internet and digital platforms, there is an opportunity to provide farmers with a simple online system where they can directly connect with customers. AgroMart is designed to make this process easy by allowing farmers to list their products and manage orders, while customers can purchase fresh products directly from farms.

The project also aims to improve transparency in pricing and provide a user-friendly system that can be easily used by both farmers and customers. Overall, the motivation is to create a simple, efficient, and reliable platform that benefits both farmers and consumers.

1d) Scope

The scope of the AgroMart project is to develop a web-based platform that connects farmers and customers for buying and selling agricultural products directly. The system allows farmers to register, upload their details and products, and manage orders, while customers can browse products, add them to the cart, and place orders easily. An admin is responsible for verifying farmers and managing the system. The platform focuses on providing a simple and user-friendly interface for all users.

The system includes basic features such as product management, order tracking, and user dashboards. It is designed to work on web browsers and provide a smooth experience for both farmers and customers.

Future improvements such as online payment integration, mobile application development, and advanced AI features are not included in the current scope and can be added later.

1e) Literature Survey / Requirement Analysis

With the growth of digital technology, many systems have been developed to improve agricultural marketing and online product selling. Various studies show that digital marketplaces help farmers reach a wider audience, reduce dependency on intermediaries, and improve price transparency. However, many existing systems are complex and not easy for farmers to use.

E-commerce platforms like Amazon and Flipkart provide convenience for customers by allowing easy browsing and purchasing, but they are not specifically designed for farmers and do not support direct farmer-to-consumer interaction. Research also highlights that the presence of multiple intermediaries increases product cost and reduces farmer profits,

making the supply chain inefficient. Although some platforms include user authentication and verification features, many still lack proper farmer verification systems, which affects trust and reliability.

From these studies, it is observed that there is a need for a simple and user-friendly platform that provides direct interaction, transparency, and trust between farmers and consumers. AgroMart is designed to address these limitations by enabling farmers to directly sell their products and allowing customers to purchase fresh produce in a transparent and efficient manner.

1e.4 Requirement Analysis:

Based on stakeholder analysis and literature review, the following requirements were identified:

Functional Requirements:

- The system shall allow farmers to register with verified identity and farm documents.
- The system shall allow admins to approve or reject farmer registrations.
- The system shall allow approved farmers to list, edit, and delete products.
- The system shall allow customers to browse, search, and filter products.
- The system shall implement a cart and checkout workflow with payment selection.
- The system shall generate order records and allow farmers to update order status.
- The system shall provide dashboards with key metrics for all user roles.

CHAPTER 2: PROJECT DESIGN

2a) Hardware, Software & Resource Requirements

Hardware Requirements:

Table 1: Hardware Requirements for Development and Deployment

Component	Minimum Specification
Processor	Intel Core i3 or higher
RAM	4 GB
Storage	256 GB
Operating System	Windows 10 / Ubuntu 20.04
Browser	Chrome 90+ / Firefox 88+

Software Requirements:

Table 2: Software Requirements

Software / Tool	Required
Frontend	React.js
Backend	Node.js and Express.js
Database	MongoDB
Development Tool	Visual Studio Code
Browser	Google Chrome or any modern web browser

2b) Dataset Design

AgroMart uses MongoDB as the database. Data is organized into four main collections with the following schema designs:

Table 3: Users Collection

Field Name	Description
name	Full name of the user
email	Email address used for login
phone	Contact phone number
password	bcrypt-hashed password
role	Role of the user in the system

Table 4: Products Collection

Field Name	Description
farmer	Reference to the farmer user
farmerProfile	Reference to farmer profile details
name	Product name (e.g., Fresh Tomatoes)
description	Detailed product description
category	Category: Vegetables, Fruits, Grains, etc.
price	Price in Indian Rupees (₹)
unit	Unit of measurement (kg, g, litre, etc.)
stock	Available stock quantity
image	Path/URL of product image

Table 5: Orders Collection

Field Name	Description
customer	Reference to the customer who placed the order
items	Array of embedded order item subdocuments
items.product	Product reference in each order item
items.name	Product name at time of order
items.price	Price per unit at time of order
items.quantity	Quantity ordered
items.farmer	Farmer who owns the ordered product
totalAmount	Total order value in ₹
deliveryAddress	Embedded address: name, phone, street, city, state, pincode
paymentMethod	Selected payment method
status	Order status: placed/accepted/shipped/delivered/cancelled
estimatedDelivery	Estimated delivery timeframe

2c) Hours Estimation

Table 6: Development Hours Estimation

Sr.	Task	Estimated Hours
1	Project Planning & Requirement Analysis	3
2	Database Schema Design (MongoDB Models)	2
3	Backend API Development (Auth, Farmer, Products, Orders, Admin)	6
4	JWT Authentication & Role-Based Middleware	2
5	File Upload Implementation (Multer for images & documents)	1
6	Admin Panel Backend APIs	4
7	UI/UX Design (CSS, Typography, Color System)	5
8	Frontend – Authentication Pages (Login, Signup)	4
9	Frontend – Farmer Registration (3-step form)	3
10	Frontend – Farmer Dashboard & Analytics	2
11	Frontend – Product Management (Add/Edit/Delete)	3
12	Frontend – Marketplace (Browse, Filter, Search)	2
13	Frontend – Product Detail & Cart Drawer	3
14	Frontend – Checkout (3-step, Multiple Payments)	2
15	Frontend – Order Confirmation & Tracking Timeline	2
16	Frontend – Admin Dashboard & Farmer Verification	4
17	Database Seeding Script	2
18	API Testing with Postman	1
19	Integration Testing (Frontend + Backend)	3
20	Bug Fixing & Code Refactoring	4
21	Documentation & Report Writing	3
	TOTAL	66

CHAPTER 3: MODULE DESCRIPTION

3a) Farmer Module

The Farmer Module allows farmers to sell their products on the AgroMart platform. It includes features such as registration, product management, and order handling.

Farmer Registration

Farmers can register by entering their personal details, farm information, and uploading required documents such as identity proof and land ownership proof. After registration, the admin verifies the details. Only approved farmers are allowed to add and sell products.

Farmer Dashboard

The dashboard provides an overview of the farmer's activity, such as:

- Total products added
- Active orders
- Total earnings
- Verification status

Product Management

Farmers can:

- Add new products with name, price, quantity, and image
- Edit product details
- Delete products
- Manage product availability

Order Management

Farmers can view and manage orders received from customers. They can update order status such as:

- Accept order
- Mark as shipped
- Mark as delivered

3b) Customer Module

The Customer Module allows users to browse and purchase products easily.

Product Browsing

Customers can:

- View products by category (vegetables, fruits, grains, etc.)
- Search for products
- View product details and farmer information

Cart and Checkout

Customers can:

- Add products to the cart
- Update quantity or remove items
- Enter delivery details
- Select payment method and place order

Order Tracking

Customers can view their order history and track order status such as:

- Placed
- Confirmed
- Delivered

3c) Admin Module

The Admin Module is the control center of the AgroMart platform, providing administrative oversight over all platform activities. The admin role is seeded directly in the database and does not require self-registration.

Admin Dashboard (Analytics):

The Admin Module manages the overall system.

Farmer Verification

Admin verifies farmer details and documents before allowing them to sell products.

User Management

Admin can view and manage all users (farmers and customers).

Order Monitoring

Admin can monitor all orders placed on the platform.

Dashboard

Admin dashboard shows basic statistics such as:

- Total users
- Total farmers
- Total orders

3d) AI-Based Features

The system includes simple AI-based features to improve user experience.

Chatbot

A basic chatbot is provided to answer common user queries such as product information, order status, and help-related questions.

Document Verification (Simulated)

During farmer registration, document verification is simulated to check uploaded files and assist admin in verification.

3e) Block Diagram with Explanation

Block Diagram Explanation

The AgroMart system follows a simple three-layer architecture consisting of the Client Layer, Server Layer, and Database Layer.

1. Client Layer (Frontend)

The client layer is the user interface of the system, developed using React.js. It is used by three types of users: Farmer, Customer, and Admin.

- Farmers can register, add products, and manage orders
- Customers can browse products, add to cart, and place orders
- Admin can verify farmers and manage the system

All user interactions are performed through the web browser.

2. Server Layer (Backend)

The server layer is developed using Node.js and Express.js. It handles all the application logic and processes user requests.

- Manages user registration and login
- Handles product and order operations

- Processes requests from frontend and sends responses
- Controls access based on user roles

3. Database Layer

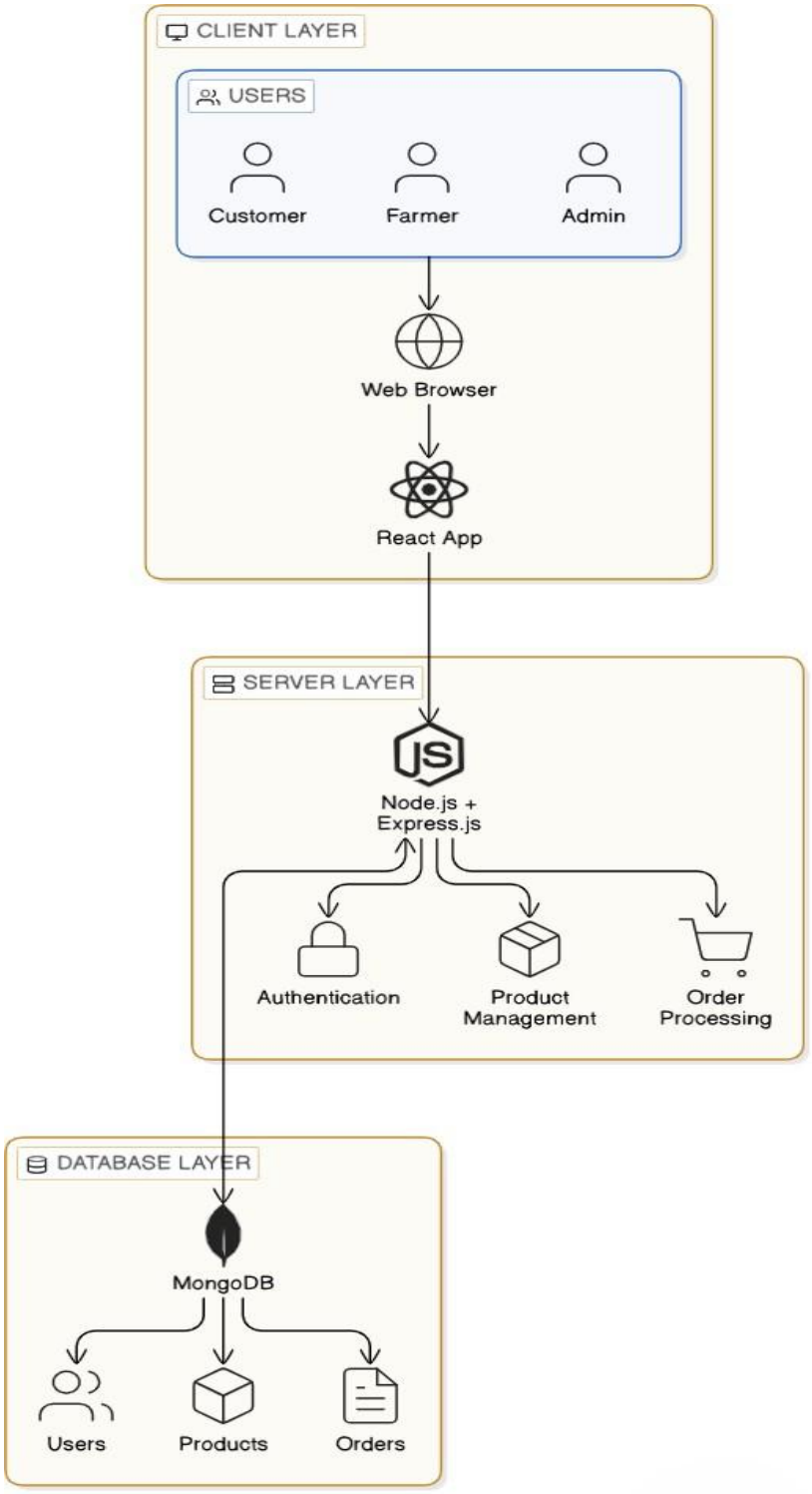
MongoDB is used as the database to store all application data.

- Stores user details (farmers, customers, admin)
- Stores product information
- Stores order details

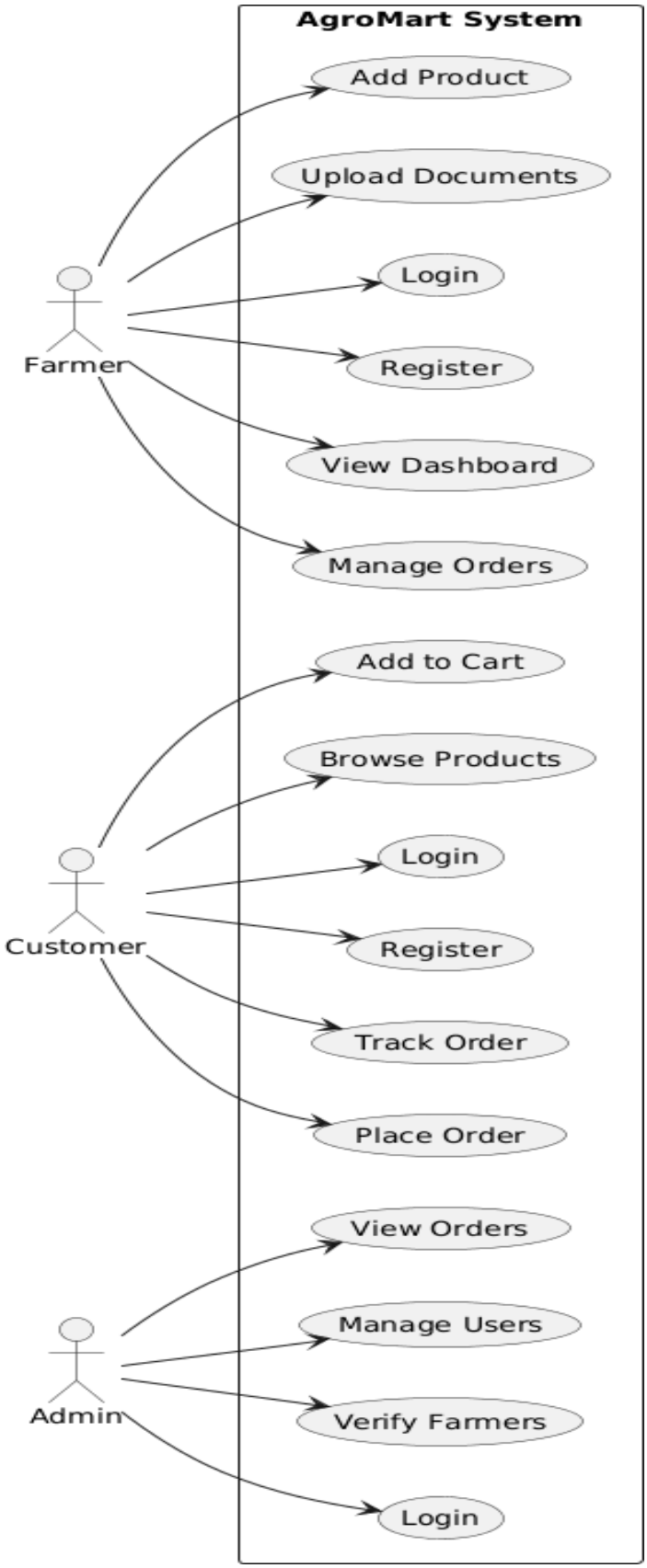
Data Flow

The working of the system can be explained as follows:

1. The user interacts with the frontend through a web browser
2. The frontend sends requests to the backend server
3. The backend processes the request and interacts with the database
4. The database returns the required data
5. The backend sends the response back to the frontend
6. The frontend displays the result to the user



a)System Architecture Diagram



b)Use Case Diagram

3f) Comparative Study

Table 10: Comparative Study of AgroMart with Existing Agricultural Platforms

Feature	AgroMart	eNAM	BigBasket	Ninjacart	Amazon Fresh
Direct Farmer-to-Consumer	Yes	Partial	No	No	No
Farmer Verification System	Yes	Yes	No	Yes	No
Consumer-Facing Marketplace	Yes	No	Yes	No	Yes
Multiple Payment Methods	Yes (5 options)	Limited	Yes	Limited	Yes
Real-time Order Tracking	Yes	No	Yes	Partial	Yes
Product Image Upload (Device)	Yes	No	N/A	No	N/A
Admin Approval Workflow	Yes	Yes	No	Yes	No
Open Source / Custom Built	Yes (MERN)	No	No	No	No
Farmer Analytics Dashboard	Yes	Partial	No	Partial	No
AI Chatbot Support	Yes (Simulated)	No	Yes	No	Yes
Indian Rupee Pricing	Yes	Yes	Yes	Yes	Yes
Free to Use for Farmers	Yes	Yes	No	No	No

CHAPTER 4: RESULTS & DISCUSSION

4a) Source Code

```
import React, { useState } from 'react';
import { Link, useNavigate, useSearchParams } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import api from '../../utils/api';
export default function Signup() {
  const [params] = useSearchParams();
  const [form, setForm] = useState({
    name: '', email: '', phone: '', password: '', confirmPassword: '',
    role: params.get('role') || 'customer'
  });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();
  const handleSubmit = async (e) => {
  e.preventDefault();
    setError('');
      if (form.password !== form.confirmPassword) return setError('Passwords do not
match');
    if (form.password.length < 6) return setError('Password must be at least 6 characters');
    setLoading(true);
    try {
      const { data } = await api.post('/auth/signup', {
        name: form.name, email: form.email, phone: form.phone,
        password: form.password, role: form.role
      });
```

```
      login(data);
      if (data.role === 'farmer') navigate('/farmer/register');
      else navigate('/marketplace');
    } catch (err) {
      setError(err.response?.data?.message || 'Signup failed');
    } finally {
      setLoading(false);
    }
  };
const mongoose = require('mongoose');

const farmerProfileSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  farmName: { type: String, required: true },
  farmLocation: { type: String, required: true },
  cropTypes: { type: String, required: true },
  farmSize: { type: String, required: true },
  govIdProof: { type: String },
  landProof: { type: String },
  bankName: { type: String, required: true },
  accountNumber: { type: String, required: true },
  ifscCode: { type: String, required: true },
  verificationStatus: {
    type: String,
    enum: ['pending', 'approved', 'rejected'],
   default: 'pending'
  },
  rejectionReason: { type: String },
```

```
  createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('FarmerProfile', farmerProfileSchema);
```

customerprofile.js-

```
import React from 'react';
import DashboardLayout from '../../components/DashboardLayout';
import { useAuth } from '../../context/AuthContext';

export default function CustomerProfile() {
  const { user } = useAuth();
            {[
              { label: 'Full Name', value: user?.name },
              { label: 'Email Address', value: user?.email },
              { label: 'Phone', value: user?.phone || 'Not provided' },
              { label: 'Member Since', value: 'AgroMart Member' },
            ].map(item => (
              <div key={item.label} className="order-detail-row">
                <span className="detail-label">{item.label}</span>
                <span className="detail-value">{item.value}</span>
              </div>
            ))}
          </div>
        </div>
      </div>
}
const mongoose = require('mongoose');

const orderItemSchema = new mongoose.Schema({
```

```
  product: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  name: String,
  image: String,
  price: Number,
  quantity: Number,
  farmer: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});

const orderSchema = new mongoose.Schema({
  customer: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  items: [orderItemSchema],
  totalAmount: { type: Number, required: true },
  deliveryAddress: {
    name: String,
    phone: String,
    street: String,
    city: String,
    state: String,
    pincode: String
  },
  paymentMethod: { type: String, default: 'Cash on Delivery' },
  status: {
    type: String,
    enum: ['placed', 'accepted', 'shipped', 'delivered', 'cancelled'],
    default: 'placed'
  },
  estimatedDelivery: { type: String, default: '3-5 business days' },
  createdAt: { type: Date, default: Date.now }
```

```
});
module.exports = mongoose.model('Order', orderSchema);
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  farmer: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  farmerProfile: { type: mongoose.Schema.Types.ObjectId, ref: 'FarmerProfile' },
  name: { type: String, required: true },
  description: { type: String },
  category: {
    type: String,
    enum: ['Vegetables', 'Fruits', 'Grains', 'Dairy', 'Spices', 'Other'],
    required: true
  },
  price: { type: Number, required: true },
  unit: { type: String, default: 'kg' },
  stock: { type: Number, required: true },
  image: { type: String },
  isAvailable: { type: Boolean, default: true },
  createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('Product', productSchema);
```

4b) Test Cases

The following test cases were executed during system testing:

Table 11: Test Cases – Authentication

TC#	Test Case	Input	Expected Output	Actual Result
TC01	Valid Customer Login	priya@example.com / customer123	JWT token returned, redirect to /marketplace	PASS
TC02	Valid Farmer Login	ramesh@farm.com / farmer123	JWT token returned, redirect to /farmer/dashboard	PASS
TC03	Valid Admin Login	admin@agromart.com / admin123	JWT token returned, redirect to /admin/dashboard	PASS
TC04	Invalid Password	priya@example.com / wrongpass	Error: Invalid credentials	PASS
TC05	Non-existent Email	notuser@x.com / anypass	Error: Invalid credentials	PASS
TC06	Duplicate Email Signup	priya@example.com (existing)	Error: Email already exists	PASS
TC07	Access Protected Route Without Token	GET /api/products/my (no token)	Error 401: No token	PASS
TC08	Customer Accessing Farmer Route	POST /api/products (customer token)	Error 403: Farmer access only	PASS

Table 12: Test Cases – Product Management

TC#	Test Case	Input	Expected Output	Actual Result
TC09	Add Product by Approved Farmer	Name: Tomatoes, Price: 45, Stock: 200	Product created, listed on marketplace	PASS
TC10	Add Product by Unapproved Farmer	Farmer with "pending" status	Error 403: Only approved farmers can add products	PASS
TC11	Browse Products with Category Filter	Category = Vegetables	Only vegetable products returned	PASS
TC12	Search Products by Name	Search query = "tomato"	Products matching "tomato" returned	PASS

TC13	Edit Product	Updated price from 45 to 50	Product price updated in database and marketplace	PASS
TC14	Delete Product	Delete Tomatoes product	Product removed from database and marketplace	PASS
TC15	Upload Product Image from Device	JPG file, 2.3 MB	Image stored, URL saved in product record	PASS
TC16	Upload Oversized Image	PNG file, 8 MB	Error: Image must be under 5MB	PASS

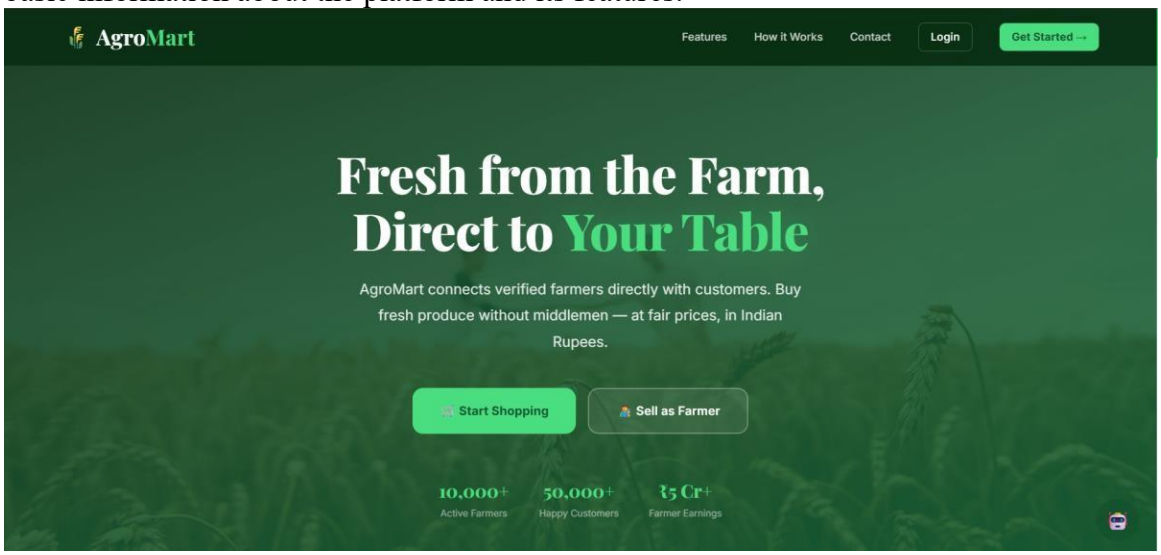
Table 13: Test Cases – Order Placement & Tracking

TC#	Test Case	Input	Expected Output	Actual Result
TC17	Place Order (COD)	Cart with 2 items, COD selected	Order created, cart cleared, confirmation page shown	PASS
TC18	Place Order (UPI)	Cart with 1 item, UPI selected	Order created with "UPI Payment" method	PASS
TC19	Order with Insufficient Stock	Product with stock=5, order qty=10	Error: Insufficient stock for [product name]	PASS
TC20	Order Confirmation Page	Navigate to /order-confirmation/:id	Order details, tracking timeline, address displayed	PASS
TC21	Farmer Accepts Order	Status change: placed → accepted	Order status updated, customer tracking updated	PASS
TC22	Farmer Marks as Shipped	Status change: accepted → shipped	Track step 3 becomes active on customer page	PASS
TC23	Farmer Marks as Delivered	Status change: shipped → delivered	All tracking steps completed (green)	PASS
TC24	Admin Approves Farmer	Farmer profile: pending → approved	Farmer can now add products, success alert on dashboard	PASS
TC25	Admin Rejects Farmer with Reason	Reason: "Invalid land document"	Rejection reason displayed on farmer dashboard	PASS

4c) Screenshots & GUI Description

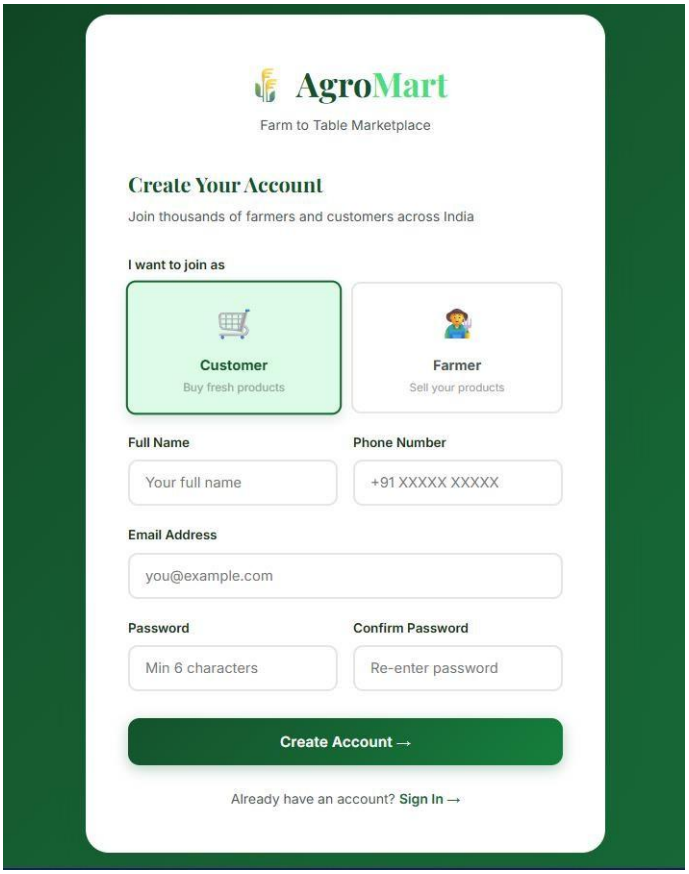
Screenshot 1: Landing Page

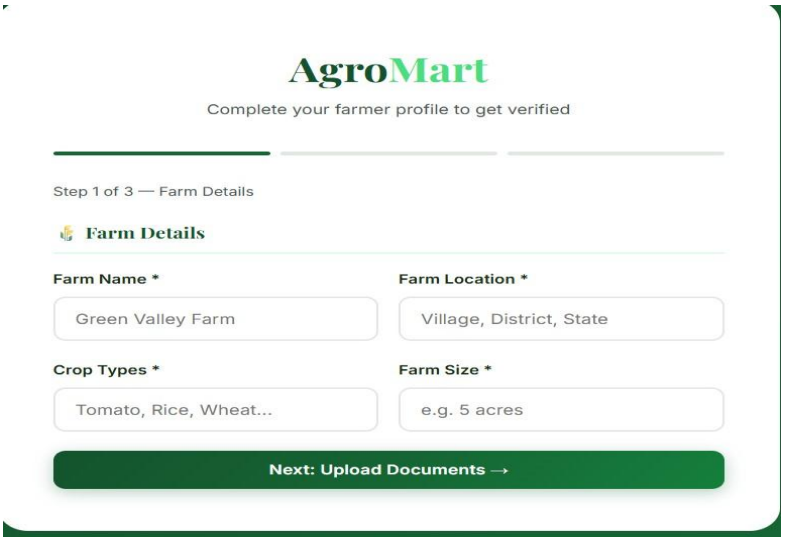
The landing page provides an introduction to the AgroMart platform. It includes the project title, a short description, and options for users to log in or register. It also displays basic information about the platform and its features.

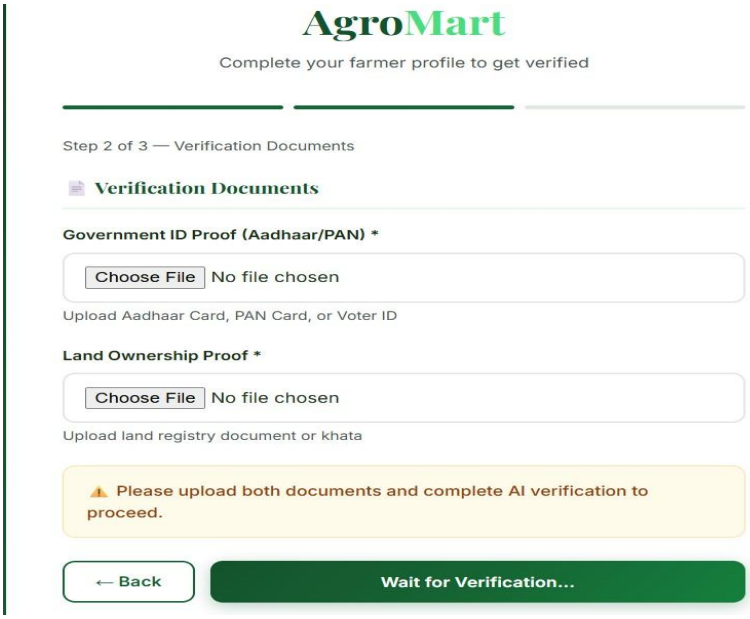


Screenshot 2: Farmer Registration Page

This page allows farmers to register by entering their personal details, farm information, and uploading required documents. The registration is divided into steps for easy data entry.

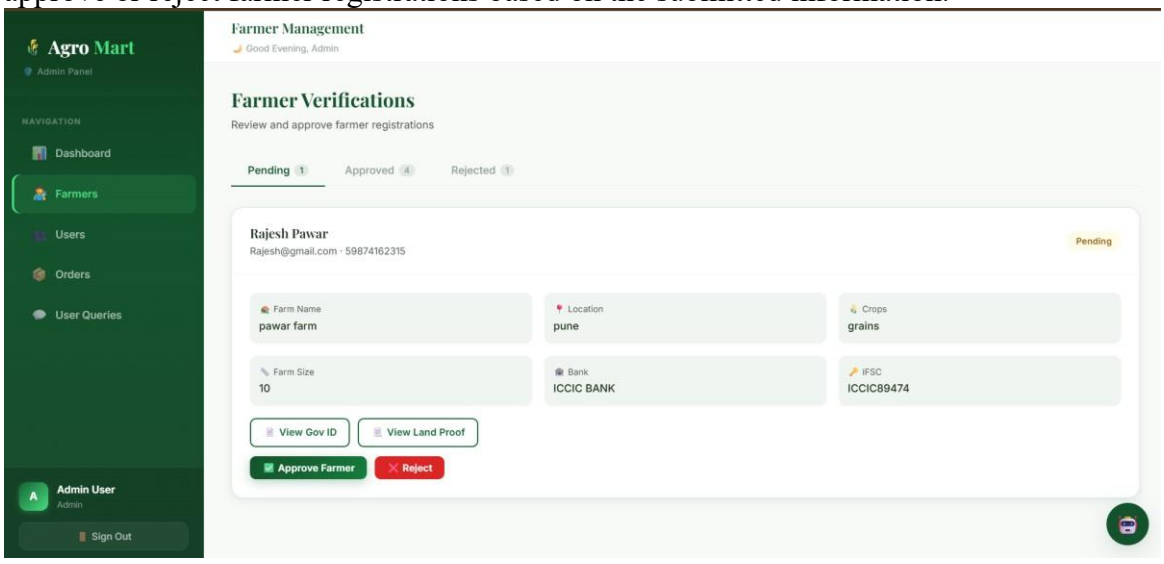






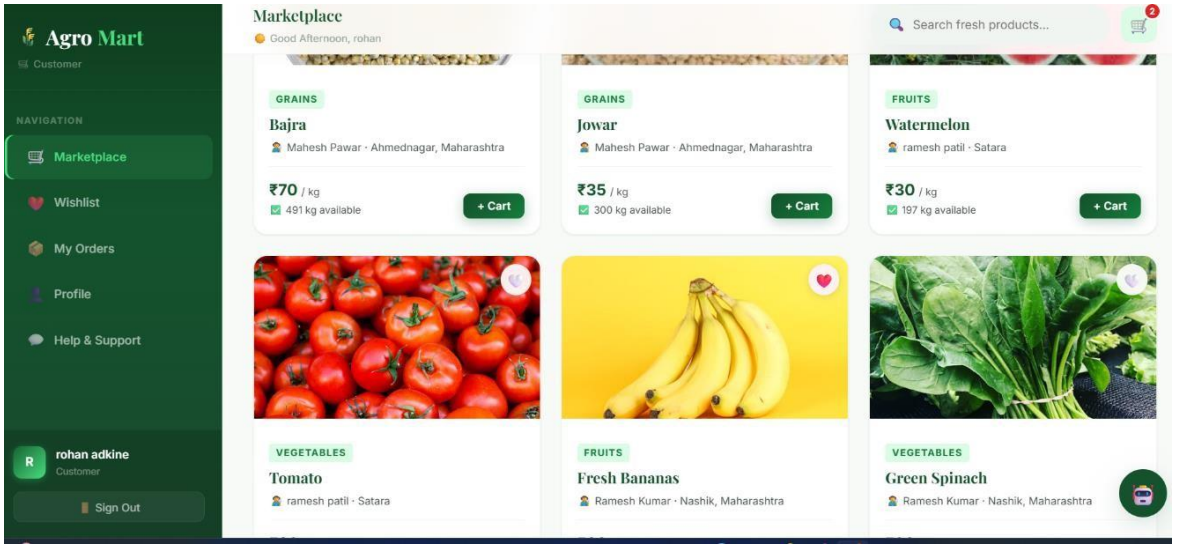
Screenshot 3: Admin – Farmer Verification

This screen shows the admin panel where farmer details are reviewed. The admin can approve or reject farmer registrations based on the submitted information.



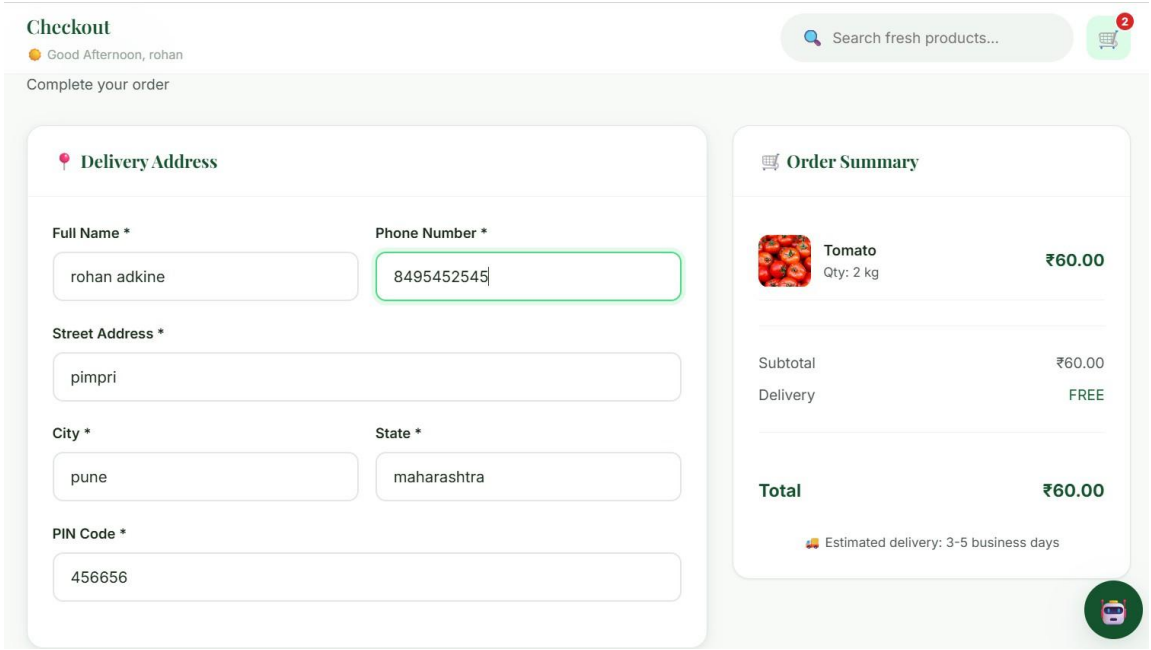
Screenshot 4: Customer Marketplace

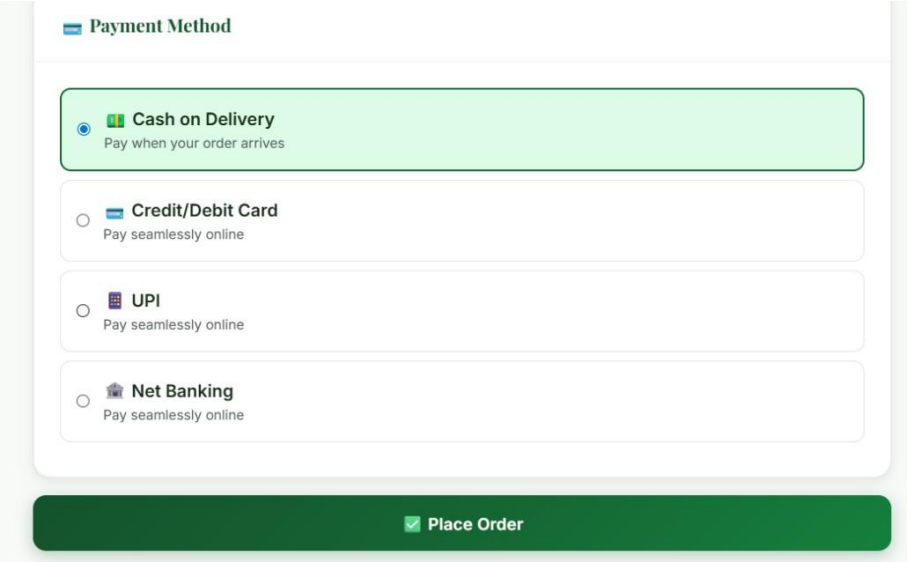
The marketplace page displays available products in a structured format. Customers can browse products by category, view product details, and add items to the cart.



Screenshot 5: Checkout Page

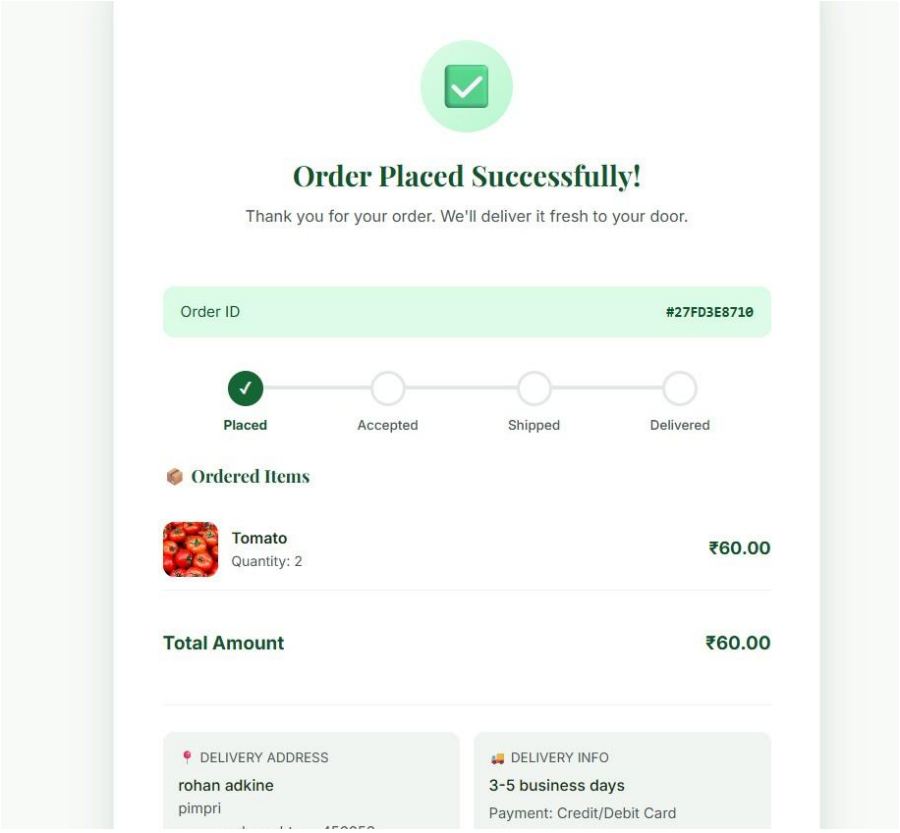
The checkout page allows customers to enter delivery details and select a payment method. It shows a summary of the selected products before placing the order.





Screenshot 6: Order Confirmation & Tracking

This page is shown after placing an order. It displays order details and allows customers to track the order status.



CHAPTER 5: CONCLUSION

5a) Conclusion

AgroMart – Farm to Table Marketplace successfully demonstrates how a web-based platform can be used to connect farmers directly with customers. The project achieves its main objective of reducing the role of intermediaries and providing a simple and transparent system for buying and selling agricultural products.

The system allows farmers to register, upload products, and manage orders, while customers can browse products, add items to the cart, and place orders . The admin module ensures that only verified farmers are allowed to sell products, which helps in maintaining trust on the platform.

The use of technologies like React.js, Node.js, Express.js, and MongoDB helped in developing a responsive and functional application. The system works smoothly and provides all the basic features required for an online agricultural marketplace.

Overall, AgroMart is a simple, user-friendly, and effective solution that can help farmers get better prices for their products and allow customers to purchase fresh produce directly from farms. The project can be further improved by adding features such as online payment integration, mobile application support, and advanced AI functionalities in the future.

5b) Future Scope

The AgroMart system can be further improved by adding more features and enhancements in the future. Some possible improvements are as follows:

1. **Online Payment Integration:**
   The system can include real payment gateway options to allow secure online transactions.
2. **Mobile Application:**
   A mobile app can be developed so that farmers and customers can easily use the platform on smartphones.
3. **Advanced Chatbot:**
   The current basic chatbot can be improved to provide better and more intelligent responses to user queries.
4. **Improved Document Verification:**
   The document verification process can be enhanced using automated systems to reduce manual work.
5. **Delivery and Tracking System:**
   A proper delivery system with real-time tracking can be implemented to improve order management.
6. **Multi-language Support:**
   The platform can support multiple languages to make it easier for farmers from different regions to use the system.
7. **Notifications System:**
   Notifications can be added to inform users about order updates and important activities.

REFERENCES

[1] Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.

[2] Brown, E. (2019). Web Development with Node and Express (2nd ed.). O'Reilly Media.

[3] Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps (2nd ed.). O'Reilly Media.

[4] Chodorow, K. (2019). MongoDB: The Definitive Guide (3rd ed.). O'Reilly Media.

[5] Mozilla Developer Network (MDN). (2024). JavaScript Reference Documentation. https://developer.mozilla.org/en-US/docs/Web/JavaScript

[6] React Documentation. (2024). React – A JavaScript library for building user interfaces. https://react.dev/

[7] MongoDB Documentation. (2024). MongoDB Manual. https://www.mongodb.com/docs/manual/

[8] Express.js Documentation. (2024). Express – Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com/

[9] Node.js Documentation. (2024). Node.js v18 API. https://nodejs.org/docs/latest-v18.x/api/

[10] JWT.io. (2024). JSON Web Tokens – Introduction. https://jwt.io/introduction

[11] National Sample Survey Office (NSSO). (2019). Situation Assessment of Agricultural Households and Land and Livestock Holdings of Households in Rural India. Ministry of Statistics and Programme Implementation, Government of India.

[12] Ministry of Agriculture & Farmers Welfare. (2023). eNAM – National Agriculture Market. https://enam.gov.in/

[13] National Informatics Centre. (2023). AgMarkNet – Agricultural Marketing Information Network. https://agmarknet.gov.in/

[14] Vite Documentation. (2024). Vite – Next Generation Frontend Tooling. https://vitejs.dev/

[15] Axios Documentation. (2024). Axios – Promise based HTTP client. https://axios-http.com/docs/intro

[16] Multer Documentation. (2024). Multer – Node.js middleware for multipart/form-data. https://github.com/expressjs/multer

[17] Mongoose Documentation. (2024). Mongoose ODM – Elegant MongoDB Object Modeling for Node.js. https://mongoosejs.com/

[18] OWASP. (2024). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

[19] Jain, R., & Sharma, P. (2021). "Digital Agricultural Marketplaces: A Study of Direct Farm-to-Consumer Models in India." International Journal of Agricultural Science and Technology, 9(2), 45–62.

[20] Kumar, A., Singh, N., & Patel, S. (2022). "MERN Stack Development: A Comprehensive Study for Full Stack Web Applications." Journal of Computer Science and Engineering Research, 14(3), 112–128.

udemy

Certificate no: UC-2d3214a5-7e9c-4b57-8673-b19b9fbdff7f
Certificate url: ude.my/UC-2d3214a5-7e9c-4b57-8673-b19b9fbdff7f
Reference Number: 0004

CERTIFICATE OF COMPLETION

MERN Stack: All You Need to Know with Practical Project

Instructors **Jahid Hiron**

PIYUSH AHIRAO

Date **Jan. 21, 2026**
Length **22 total hours**

Plagiarism Scan Report

3% Plagiarism	9% Exact Match
	0% Partial Match
97% Unique	

Words	1000
Characters	6552
Sentences	361
Paragraphs	832
Read Time	24 minute(s)
Speak Time	32 minute(s)

Content Checked For Plagiarism

Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering (PCCoE)
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University(SPPU) ISO 21001:2018 Certified by TUV SUD
Sustainability Development Goals Mapping
Department: Computer Engineering A.Y.: 2025-26 Sem: 2
Subject: Full Stack Development Lab.

Mini Project Title: AgroMart – Farm to Table Marketplace. PRN: 123B1B077.
Name: Piyush Ahirao. Class and Div: T.Y. B. Lab Location: 6218LA
Sr. No.
SDG No.
SDG
Description
1
SDG 2
Zero Hunger
AgroMart connects farmers directly with consumers, improving access to fresh and quality food.
2
SDG 8
Decent Work & Economic Growth
It helps farmers increase their income by providing a platform for direct selling without intermediaries.
3
SDG 9
Industry, Innovation & Infrastructure
The system uses modern web technologies to build an efficient digital agricultural marketplace.
4
SDG 10
Reduced Inequalities
It provides equal opportunities for small and local farmers to reach a wider market.
5
SDG 12
Responsible Consumption & Production
AgroMart encourages direct sales, reducing food wastage and promoting better consumption practices.

Mr.Shailesh B. Galande